

Aula 06: Microserviços na Prática — Comunicação, Gateways e o Desafio da Consistência

1. Introdução: A Transição do Monólito para Microserviços no Desenvolvimento Full Cycle

Sejam bem-vindos à nossa sexta aula de Tópicos Avançados em Computação. Na aula anterior, demos um passo gigante ao dominar o Docker e o Docker Compose, aprendendo a “empacotar” nossa aplicação e suas dependências em containers. Até agora, estávamos lidando com o que chamamos de **Monólito Containerizado** — uma única aplicação que faz tudo, mas que roda dentro de um container. Hoje, vamos quebrar esse monólito e entrar de cabeça na **Arquitetura de Microserviços** [1].

Para um desenvolvedor **Full Cycle**, a transição para microserviços não é apenas uma escolha tecnológica, é uma resposta à necessidade de escala e agilidade. Em um monólito, se o módulo de Pagamentos precisa de mais recursos, você é obrigado a escalar a aplicação inteira. Em microserviços, cada funcionalidade (Pedidos, Catálogo, Pagamentos) torna-se um serviço independente, com seu próprio ciclo de vida, banco de dados e tecnologia. No entanto, como dizem: “Não existe almoço grátis”. Ao ganhar escalabilidade e independência, ganhamos também uma complexidade enorme na comunicação e na consistência dos dados [2].

1.1. Por que Microserviços? (O Diferencial no Mercado)

O mercado de trabalho valoriza o profissional que entende quando e como usar microserviços. Não se trata de seguir uma moda, mas de resolver problemas reais de grandes empresas. As principais vantagens que exploraremos nesta fase do curso são:

- **Escalabilidade Granular:** Podemos escalar apenas o serviço que está sob carga (ex: o Catálogo durante a Black Friday).
- **Independência de Deploy:** Uma alteração no serviço de Notificações não exige o re-deploy de toda a plataforma.
- **Isolamento de Falhas:** Se o serviço de Pagamentos cair, o cliente ainda pode navegar no Catálogo e adicionar itens ao carrinho.
- **Liberdade Tecnológica:** Podemos usar .NET para o serviço de Pedidos e Python para um serviço de Recomendação baseado em IA.

Característica	Monólito	Microserviços
Complexidade	Baixa (inicialmente)	Alta (desde o início)
Deploy	Tudo ou nada	Independente por serviço
Escalabilidade	Vertical (mais hardware)	Horizontal (mais instâncias do serviço)
Banco de Dados	Único e centralizado	Descentralizado (um por serviço)
Comunicação	Chamadas de método em memória	Chamadas de rede (HTTP, Mensageria)

2. Comunicação Síncrona: O Diálogo via HTTP entre Serviços

Em uma arquitetura distribuída, os serviços precisam “conversar” para realizar uma tarefa de negócio. Quando um cliente faz um pedido, o **Serviço de Pedidos** precisa perguntar ao **Serviço de Catálogo** se o produto existe e qual o seu preço atual. A forma mais comum e direta de realizar essa conversa é através da **Comunicação Síncrona**, geralmente utilizando o protocolo **HTTP** com o padrão **REST** [3].

2.1. O Protocolo HTTP como Padrão de Fato

O HTTP é o “esperanto” da web. Quase qualquer tecnologia moderna sabe falar HTTP, o que o torna a escolha natural para a integração entre microsserviços. Na comunicação síncrona, o serviço que faz a requisição (Cliente) fica aguardando a resposta do serviço que recebe a requisição (Servidor). É como uma ligação telefônica: você fala, a outra pessoa ouve e responde imediatamente.

No entanto, essa simplicidade tem um custo: o **Acoplamento Temporal**. Se o Serviço de Catálogo estiver lento ou fora do ar, o Serviço de Pedidos também ficará lento ou falhará, pois ele depende da resposta imediata para continuar o seu processamento [4].

2.2. Implementando o `IHttpClientFactory` no .NET

Muitos desenvolvedores cometem o erro clássico de instanciar o `HttpClient` dentro de um bloco `using` ou a cada requisição (`new HttpClient()`). Isso pode levar à **exaustão de sockets**, pois o sistema operacional demora para liberar as conexões TCP fechadas.

No .NET moderno, a prática recomendada é usar o **`IHttpClientFactory`**. Ele gerencia o ciclo de vida dos handlers de mensagens HTTP, reutilizando conexões de forma eficiente e permitindo a configuração centralizada de políticas de resiliência [5].

```
// Exemplo de configuração no Program.cs
builder.Services.AddHttpClient<ICatalogoService, CatalogoService>(client =>
{
    client.BaseAddress = new Uri("http://catalogo-api");
});
```

```
// Uso no serviço
public class CatalogoService : ICatalogoService
{
    private readonly HttpClient _httpClient;

    public CatalogoService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<ProdutoDTO> ObterProdutoAsync(int id)
    {

```

```

        var response = await _httpClient.GetAsync($"/api/produtos/{id}");
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<ProdutoDTO>();
    }
}

```

2.3. Resiliência com Polly: O Cinto de Segurança da Comunicação

Como a rede é inerentemente instável, não podemos simplesmente “torcer” para que a chamada HTTP funcione. Precisamos de estratégias de resiliência. A biblioteca **Polly** é o padrão ouro no ecossistema .NET para isso. Ela nos permite implementar padrões como:

- **Retry (Retentativa):** Tentar novamente a chamada se houver uma falha temporária (ex: um timeout rápido).
- **Circuit Breaker (Disjuntor):** “Abrir o circuito” e parar de fazer chamadas para um serviço que está falhando consistentemente, dando tempo para ele se recuperar e evitando o efeito cascata de falhas no sistema [6].

“Em microsserviços, falhas são inevitáveis. O diferencial do profissional sênior é projetar o sistema para que ele falhe com elegância e se recupere automaticamente.”

Na próxima seção, veremos como organizar o acesso a esses múltiplos serviços de forma que o cliente final não precise conhecer a complexidade interna da nossa arquitetura: o **API Gateway**.

3. API Gateway: A Porta de Entrada do Sistema

Imagine que o seu cliente (um aplicativo mobile ou um site) precise fazer um pedido. Ele teria que saber o endereço IP do Serviço de Pedidos, do Serviço de Catálogo, do Serviço de Pagamentos e do Serviço de Notificações. Além disso, ele teria que lidar com diferentes protocolos, autenticação em cada serviço e a complexidade de agregar dados de múltiplos lugares. Expor seus microsserviços diretamente ao mundo exterior é um erro de segurança e arquitetura [7].

É aqui que entra o **API Gateway**. Ele atua como um ponto de entrada único para todas as requisições externas. O cliente faz uma chamada para o Gateway, e o Gateway decide para qual microsserviço interno essa requisição deve ser encaminhada. Ele funciona como o “recepcionista” de um grande prédio comercial: você diz com quem quer falar, e ele te direciona para a sala correta [8].

3.1. O que é um API Gateway? (Roteamento, Agregação e Segurança)

O API Gateway resolve diversos problemas de uma só vez:

- **Roteamento:** Encaminha a requisição para o serviço correto (ex: /api/pedidos vai para o Serviço de Pedidos).
- **Agregação de Dados:** Pode fazer chamadas para múltiplos serviços e retornar uma única resposta consolidada para o cliente.
- **Segurança:** Centraliza a autenticação e autorização (ex: valida o token JWT uma única vez no Gateway).

- **Abstração:** O cliente não precisa saber que o sistema é composto por 50 microsserviços; para ele, existe apenas uma API.
- **Políticas de Tráfego:** Implementa Rate Limiting (limite de requisições) e Caching de forma centralizada.

3.2. YARP (Yet Another Reverse Proxy) vs. Ocelot no Ecosistema .NET

No mundo .NET, temos duas opções principais para implementar um API Gateway:

- **Ocelot:** Um framework clássico e muito popular, fácil de configurar via arquivo JSON. É excelente para cenários simples e médios.
- **YARP (Yet Another Reverse Proxy):** Criado pela própria Microsoft, o YARP é um proxy reverso altamente performático e extensível, construído sobre o ASP.NET Core. Ele é a recomendação atual para novos projetos que buscam alta performance e integração nativa com o .NET [9].

Característica	Ocelot	YARP
Performance	Boa	Excelente (Nativo ASP.NET Core)
Configuração	JSON (Estático)	Código C# ou JSON (Dinâmico)
Extensibilidade	Média	Alta (Middleware nativo)
Suporte Microsoft	Comunidade	Oficial Microsoft

3.3. Configurando o YARP para Roteamento Simples

Configurar o YARP é surpreendentemente simples. No seu projeto de Gateway (uma Web API vazia), você define as **Rotas** (o que o cliente chama) e os **Clusters** (para onde o Gateway deve mandar a chamada) [10].

// appsettings.json do Gateway

```
{
  "ReverseProxy": {
    "Routes": {
      "pedidos-route": {
        "ClusterId": "pedidos-cluster",
        "Match": { "Path": "/api/pedidos/{**catch-all}" }
      },
      "catalogo-route": {
        "ClusterId": "catalogo-cluster",
        "Match": { "Path": "/api/catalogo/{**catch-all}" }
      }
    },
    "Clusters": {
      "pedidos-cluster": {
        "Destinations": { "destination1": { "Address": "http://pedidos-api/" }
        }
      },
      "catalogo-cluster": {
        "Destinations": { "destination1": { "Address": "http://catalogo-api/" }
        }
      }
    }
  }
}
```

```
}  
  }  
}
```

4. O Pesadelo da Consistência em Sistemas Distribuídos

Em um monólito com um único banco de dados, garantir a consistência é fácil: você abre uma transação SQL, faz todas as alterações e dá um COMMIT. Se algo falhar, o banco faz um ROLLBACK e tudo volta ao estado original. Isso é o que chamamos de **Transações ACID** (Atomicidade, Consistência, Isolamento e Durabilidade) [11].

Em microsserviços, cada serviço tem seu próprio banco de dados. Quando um pedido é criado, o Serviço de Pedidos grava no seu banco, e o Serviço de Catálogo precisa atualizar o estoque no banco dele. Não existe uma transação SQL que cubra dois bancos de dados diferentes em servidores diferentes. Se o primeiro gravar e o segundo falhar, temos um problema de **Inconsistência de Dados** [12].

4.1. Teorema CAP: O Trilema do Desenvolvedor

O **Teorema CAP** afirma que, em um sistema distribuído, é impossível garantir simultaneamente as três propriedades:

- **C (Consistency - Consistência):** Todos os nós veem os mesmos dados ao mesmo tempo.
- **A (Availability - Disponibilidade):** Toda requisição recebe uma resposta (sucesso ou falha).
- **P (Partition Tolerance - Tolerância a Partição):** O sistema continua funcionando mesmo se houver falhas na rede entre os nós.

Como a rede sempre pode falhar (P), você é obrigado a escolher entre Consistência (C) ou Disponibilidade (A). Na maioria dos sistemas de microsserviços modernos, escolhemos a Disponibilidade e aceitamos a **Consistência Eventual** [13].

4.2. Consistência Forte vs. Consistência Eventual

- **Consistência Forte:** O dado é atualizado em todos os lugares instantaneamente. É o modelo do banco de dados relacional tradicional.
- **Consistência Eventual:** O sistema garante que, se nenhuma nova atualização for feita, eventualmente todos os nós terão o dado mais recente. Durante um curto período, um usuário pode ver um dado antigo enquanto a atualização se propaga pelo sistema.

4.3. Introdução ao Saga Pattern (Visão Geral)

Para lidar com falhas em transações que envolvem múltiplos microsserviços, usamos o **Saga Pattern**. Uma Saga é uma sequência de transações locais. Se uma etapa falhar, a Saga executa **Transações Compensatórias** para desfazer o que foi feito nas etapas anteriores. Por exemplo: se o pagamento falhar, a Saga chama o Serviço de Pedidos para cancelar o pedido e o Serviço de Catálogo para devolver o item ao estoque [14].

“O segredo de uma arquitetura de microsserviços resiliente não é evitar falhas, mas saber como se recuperar delas de forma consistente.”

Na próxima seção, colocaremos tudo isso em prática, evoluindo nossa Plataforma de Pedidos para um cenário multi-serviço com API Gateway.

5. Prática Diária: Evoluindo a Plataforma de Pedidos

Chegou o momento de colocar a mão na massa! Vamos dar continuidade ao projeto da nossa **Plataforma de Gestão de Pedidos**. O objetivo hoje é evoluir o monólito containerizado da aula anterior para uma arquitetura de microsserviços real, com dois serviços independentes e um API Gateway centralizando o acesso [15].

Cenário da Atividade

Você deve criar o **Service B (Catálogo)**, implementar a comunicação HTTP entre o **Service A (Pedidos)** e o Catálogo, e configurar o **YARP API Gateway** para rotear as requisições.

Passo 1: Criar a Web API de Catálogo

Crie um novo projeto Web API chamado GestaoPedidos.Catalogo. Este serviço será responsável por gerenciar os produtos e seus preços.

```
// Exemplo de Controller no Serviço de Catálogo
[ApiController]
[Route("api/produtos")]
public class ProdutosController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        // Simulação de busca no banco de dados do Catálogo
        return Ok(new { Id = id, Nome = "Notebook", Preco = 2500.00m });
    }
}
```

Passo 2: Implementar a Chamada HTTP no Serviço de Pedidos

No projeto GestaoPedidos.API (Pedidos), adicione o `IHttpClientFactory` e crie um serviço para consultar o Catálogo.

```
// No Program.cs do Serviço de Pedidos
builder.Services.AddHttpClient("Catalogo", client =>
{
    client.BaseAddress = new Uri("http://catalogo-api/");
});

// No Controller de Pedidos
[HttpPost]
public async Task<IActionResult> CriarPedido([FromBody] PedidoRequest request,
[FromServices] IHttpClientFactory clientFactory)
```

```

{
    var client = clientFactory.CreateClient("Catalogo");
    var response = await client.GetAsync($"api/produtos/{request.ProdutoId}");

    if (!response.IsSuccessStatusCode)
        return BadRequest("Produto não encontrado no Catálogo.");

    // Lógica de criação do pedido...
    return CreatedAtAction(nameof(GetById), new { id = 1 }, new { Id = 1 });
}

```

Passo 3: Configurar o YARP API Gateway

Crie um novo projeto Web API vazio chamado GestaoPedidos.Gateway e instale o pacote Yarp.ReverseProxy.

```

// No Program.cs do Gateway
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

var app = builder.Build();
app.MapReverseProxy();
app.Run();

```

Configure o appsettings.json do Gateway para rotear /api/pedidos para o serviço de Pedidos e /api/catalogo para o serviço de Catálogo.

Passo 4: Atualizar o Docker Compose

Atualize o seu arquivo docker-compose.yml para incluir os três serviços e garantir que eles estejam na mesma rede.

```
version: '3.8'
```

```

services:
  gateway:
    build: ./GestaoPedidos.Gateway
    ports:
      - "8000:80"
    depends_on:
      - pedidos-api
      - catalogo-api

  pedidos-api:
    build: ./GestaoPedidos.API
    environment:
      - ConnectionStrings__DefaultConnection=Server=db-pedidos;...
    depends_on:
      - db-pedidos

```

```
catalogo-api:
  build: ./GestaoPedidos.Catalogo

db-pedidos:
  image: mcr.microsoft.com/mssql/server:2022-latest
  # ... configurações do banco
```

Passo 5: Testar o Fluxo Completo

No terminal, execute `docker-compose up -d`. Agora, em vez de acessar as APIs diretamente, acesse através do Gateway: - GET `http://localhost:8000/api/catalogo/produtos/1` - POST `http://localhost:8000/api/pedidos`

Verifique os logs com `docker-compose logs -f` para ver a comunicação acontecendo entre os serviços.

6. Glossário de Termos Técnicos

Para ajudar vocês a navegarem nesse mar de siglas e conceitos, preparei este glossário com os termos que todo desenvolvedor Full Cycle deve dominar:

- **API Gateway:** Ponto de entrada único para um sistema de microsserviços.
 - **BFF (Backend for Frontend):** Padrão onde um Gateway é criado especificamente para um tipo de cliente (ex: Mobile vs. Web).
 - **Circuit Breaker:** Padrão de resiliência que interrompe chamadas para um serviço que está falhando.
 - **Comunicação Síncrona:** Modelo onde o cliente aguarda a resposta imediata do servidor (ex: HTTP).
 - **Consistência Eventual:** Modelo onde os dados podem ficar temporariamente inconsistentes, mas eventualmente se tornam iguais em todos os nós.
 - **IHttpClientFactory:** Ferramenta do .NET para gerenciar instâncias de `HttpClient` de forma eficiente.
 - **Ocelot:** Framework popular de API Gateway para .NET.
 - **Polly:** Biblioteca de resiliência e tratamento de falhas transitórias para .NET.
 - **Reverse Proxy:** Servidor que encaminha requisições de clientes para um ou mais servidores internos.
 - **Saga Pattern:** Padrão para gerenciar transações distribuídas e consistência entre microsserviços.
 - **Teorema CAP:** Princípio que define os limites de consistência, disponibilidade e tolerância a partição em sistemas distribuídos.
 - **YARP (Yet Another Reverse Proxy):** Proxy reverso de alta performance da Microsoft para .NET.
-

7. Referências Bibliográficas

Para aprofundar seus conhecimentos, recomendo a leitura das seguintes obras e recursos:

1. **NEWMAN, Sam.** *Building Microservices: Designing Fine-Grained Systems*. 2. ed. O'Reilly Media, 2021. (A “bíblia” dos microsserviços).
2. **RICHARDSON, Chris.** *Microservices Patterns: With examples in Java*. Manning Publications, 2018. (Excelente para entender padrões como Saga e API Gateway).
3. **MICROSOFT.** *Communication in a microservice architecture*. Disponível em: <https://learn.microsoft.com/pt-pt/dotnet/architecture/microservices/architect-microservice-container-applications/communication-in-microservice-architecture>. Acesso em: 13 mar. 2026.
4. **MICROSOFT.** *Implement API Gateways with Ocelot*. Disponível em: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/multi-container-microservice-net-applications/implement-api-gateways-with-ocelot>. Acesso em: 13 mar. 2026.
5. **YARP.** *YARP Documentation*. Disponível em: <https://microsoft.github.io/reverse-proxy/>. Acesso em: 13 mar. 2026.
6. **POLLY.** *Polly: Resilience and transient-fault-handling library*. Disponível em: <https://github.com/App-vNext/Polly>. Acesso em: 13 mar. 2026.
7. **FOWLER, Martin.** *Microservices*. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 13 mar. 2026.
8. **JOVANOVIĆ, Milan.** *Implementing an API Gateway For Microservices With YARP*. Disponível em: <https://www.milanjovanovic.tech/blog/implementing-an-api-gateway-for-microservices-with-yarp>. Acesso em: 13 mar. 2026.
9. **MEDIUM.** *API Gateway in .Net Microservice Architecture*. Disponível em: <https://dotnetfullstackdev.medium.com/api-gateway-in-net-microservice-architecture-411cdf52c22d>. Acesso em: 13 mar. 2026.
10. **DEV.TO.** *Saga Pattern: Consistência de Dados em Microsserviços de Verdade*. Disponível em: https://dev.to/filipi_firmino_dev/saga-pattern-consistencia-de-dados-em-microsservicos-de-verdade-4h34. Acesso em: 13 mar. 2026.
11. **LINKEDIN.** *Módulo 4: Transações Distribuídas e o Padrão SAGA*. Disponível em: <https://pt.linkedin.com/pulse/module-4-distributed-transactions-saga-pattern-raj-kothari-6vzic>. Acesso em: 13 mar. 2026.
12. **MELLO, Elisandro.** *Atomicidade em sistemas distribuídos: 2PC explicado de forma simples*. Medium, 2025. Disponível em: <https://elisandromello.medium.com/atomicidade-em-sistemas-distribu%C3%ADdos-2pc-explicado-de-forma-simples-65012e991d9d>. Acesso em: 13 mar. 2026.
13. **SPEAKER DECK.** *gRPC: Conhecendo novo modelo de comunicação entre microsserviços*. Disponível em: <https://speakerdeck.com/neiesc/grpc-conhecendo-novo-modelo-de-comunicacao-entre-microsservicos>. Acesso em: 13 mar. 2026.
14. **SOMMERVILLE, Ian.** *Engenharia de Software*. 10. ed. Pearson, 2018. (Capítulo sobre arquiteturas distribuídas).

15. **DOCKER.** *Docker Compose for .NET Developers.* Medium, [s.d.]. Disponível em: https://medium.com/@compileandconquer/docker-compose-for-net-56d7cbce964e[https://medium.com/@compileandconquer/docker-compose-for-net-56d7cbce964e]. Acesso em: 13 mar. 2026.